

Transfer Learning Image Classification

CODE:-

Part 1: The Complete Training Notebook Script

```
import os
import time
import copy
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import numpy as np
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms, models

#
=====

=====
# 1. SETUP & HYPERPARAMETERS
#
=====

=====
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

BATCH_SIZE = 32
NUM_EPOCHS = 5
LEARNING_RATE = 0.001
NUM_CLASSES = 3 # Subset: Airplane (0), Automobile (1), Bird (2)
```

```

#
=====

# 2. DATA AUGMENTATION & LOADING (CIFAR-10 SUBSET)
#
=====

print("\nPreparing datasets and applying data augmentations...")

# ResNet expects 224x224 images normalized with ImageNet statistics
train_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

test_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

# Download full CIFAR-10 dataset
full_train_dataset = datasets.CIFAR10(root='./data', train=True,
download=True, transform=train_transforms)
full_test_dataset = datasets.CIFAR10(root='./data', train=False,
download=True, transform=test_transforms)

# Filter for classes 0, 1, 2 (Airplane, Automobile, Bird)
def get_subset_indices(dataset, num_classes=3):
    return [i for i, label in enumerate(dataset.targets) if label < num_classes]

train_indices = get_subset_indices(full_train_dataset, NUM_CLASSES)

```

```
test_indices = get_subset_indices(full_test_dataset, NUM_CLASSES)
```

```
train_dataset = Subset(full_train_dataset, train_indices)
```

```
test_dataset = Subset(full_test_dataset, test_indices)
```

```
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE,  
shuffle=True, num_workers=2)
```

```
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE,  
shuffle=False, num_workers=2)
```

```
print(f"Training samples: {len(train_dataset)} | Test samples:  
{len(test_dataset)}")
```

```
#
```

```
=====
```

```
# 3. TRANSFER LEARNING MODEL CONFIGURATION
```

```
#
```

```
=====
```

```
print("\nInitializing Pre-trained ResNet18...")
```

```
# Load weights using the modern weights parameter
```

```
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
```

```
# Freeze backbone convolutional layers so we don't destroy pre-trained  
features
```

```
for param in model.parameters():
```

```
    param.requires_grad = False
```

```
# Replace the final fully connected layer (Classifier) with a new custom  
head
```

```
num_fts = model.fc.in_features
```

```
model.fc = nn.Linear(num_fts, NUM_CLASSES)
```

```
model = model.to(device)
```

```
# Loss function and Optimizer (Only optimize parameters of the final
layer)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.fc.parameters(), lr=LEARNING_RATE)
```

```
#
```

```
=====
=====
```

```
# 4. TRAINING & EVALUATION LOOP
```

```
#
```

```
=====
=====
```

```
print("\nStarting Training...")
history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': []}
best_model_wts = copy.deepcopy(model.state_dict())
best_acc = 0.0
```

```
for epoch in range(NUM_EPOCHS):
    print(f'Epoch {epoch + 1}/{NUM_EPOCHS}')
    print('-' * 10)
```

```
# Each epoch has a training and validation phase
```

```
for phase in ['train', 'val']:
    if phase == 'train':
        model.train()
        dataloader = train_loader
    else:
        model.eval()
        dataloader = test_loader
```

```
running_loss = 0.0
running_corrects = 0
```

```
# Iterate over data batches
for inputs, labels in dataloader:
```

```

inputs = inputs.to(device)
labels = labels.to(device)

optimizer.zero_grad()

# Forward pass
with torch.set_grad_enabled(phase == 'train'):
    outputs = model(inputs)
    _, preds = torch.max(outputs, 1)
    loss = criterion(outputs, labels)

# Backward pass & optimize only if in training phase
if phase == 'train':
    loss.backward()
    optimizer.step()

running_loss += loss.item() * inputs.size(0)
running_corrects += torch.sum(preds == labels.data)

epoch_loss = running_loss / len(dataloader.dataset)
epoch_acc = running_corrects.double() / len(dataloader.dataset)

print(f'{phase.capitalize()} Loss: {epoch_loss:.4f} Acc:
{epoch_acc:.4f}')

# Save history metrics
if phase == 'train':
    history['train_loss'].append(epoch_loss)
    history['train_acc'].append(epoch_acc.cpu().item())
else:
    history['val_loss'].append(epoch_loss)
    history['val_acc'].append(epoch_acc.cpu().item())

# Deep copy the best weights
if epoch_acc > best_acc:

```

```

        best_acc = epoch_acc
        best_model_wts = copy.deepcopy(model.state_dict())
    print()

print(f'Training complete! Best Val Acc: {best_acc:4f}')

# Load best model weights before saving
model.load_state_dict(best_model_wts)

#
=====

=====

# 5. SAVE MODEL & PLOT METRICS
#
=====

=====

# Save weights and architecture structure via state_dict
MODEL_PATH = "resnet18_cifar_subset.pth"
torch.save(model.state_dict(), MODEL_PATH)
print(f'Model saved successfully to '{MODEL_PATH}''')

# Generate training curve charts
epochs_range = range(1, NUM_EPOCHS + 1)
plt.figure(figsize=(12, 5))

# Plot Loss Curves
plt.subplot(1, 2, 1)
plt.plot(epochs_range, history['train_loss'], label='Train Loss', marker='o')
plt.plot(epochs_range, history['val_loss'], label='Val Loss', marker='o')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.grid(True)

```

```
# Plot Accuracy Curves
plt.subplot(1, 2, 2)
plt.plot(epochs_range, history['train_acc'], label='Train Acc', marker='o')
plt.plot(epochs_range, history['val_acc'], label='Val Acc', marker='o')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Training and Validation Accuracy')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```

Part 2: Instructions to Run Inference

Once the script completes, a model weights file named `resnet18_cifar_subset.pth` will be generated in your working directory. You can use the following clean Python snippet to run standalone inferences on a local raw target image.

1. Requirements for Inference Script

Make sure the image file you want to test is in the same directory (e.g., sample an image file and label it `test_image.jpg`)

```
import torch
import torch.nn as nn
from torchvision import models, transforms
from PIL import Image

# 1. Define configurations & Class Labels mappings
DEVICE = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
LABELS = {0: "Airplane", 1: "Automobile", 2: "Bird"}
```

```
MODEL_PATH = "resnet18_cifar_subset.pth"
IMAGE_PATH = "test_image.jpg" # Replace with your image file path
```

```
# 2. Reconstruct Model Structure
```

```
model = models.resnet18()
num_fts = model.fc.in_features
model.fc = nn.Linear(num_fts, len(LABELS)) # Custom 3-class linear head
model.load_state_dict(torch.load(MODEL_PATH,
map_location=DEVICE))
model = model.to(DEVICE)
model.eval()
```

```
# 3. Define the deterministic validation transform pipeline
```

```
inference_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
```

```
# 4. Load, preprocess, and execute forward pass on image
```

```
try:
```

```
    img = Image.open(IMAGE_PATH).convert('RGB')
```

```
    input_tensor = inference_transforms(img).unsqueeze(0).to(DEVICE) #
```

```
Add batch dimension
```

```
with torch.no_grad():
```

```
    outputs = model(input_tensor)
```

```
    probabilities = torch.nn.functional.softmax(outputs[0], dim=0)
```

```
    confidence, predicted_idx = torch.max(probabilities, 0)
```

```
print(f"\n--- Inference Result ---")
```

```
print(f'Predicted Class: {LABELS[predicted_idx.item()]}\n')
```

```
print(f'Confidence Score: {confidence.item() * 100:.2f}%\n')
```

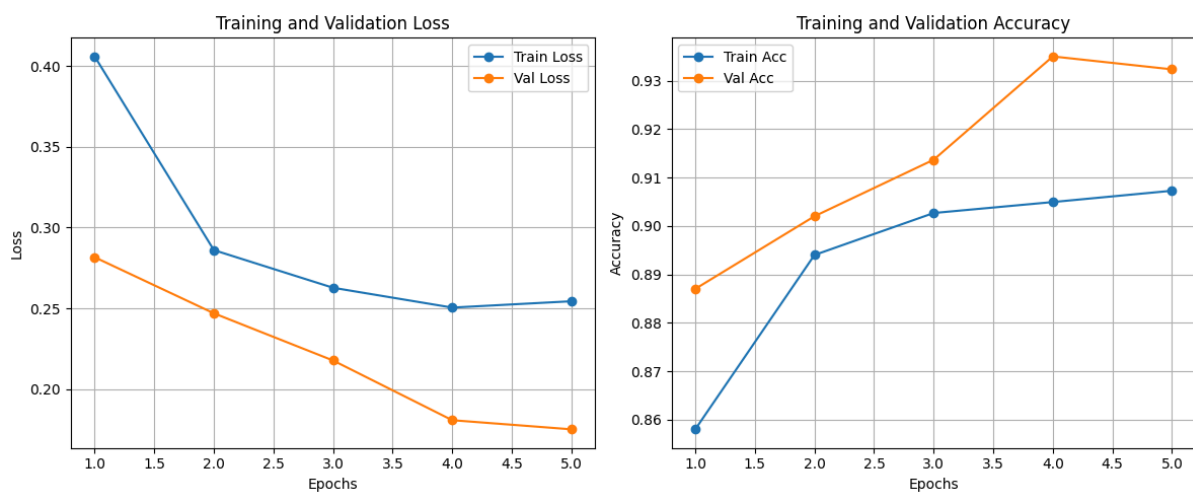
except FileNotFoundError:

```
print(f'Error: Could not find image file at '{IMAGE_PATH}'. Please  
check your file name path.")
```

JUPYTER NOTEBOOK LINK:-

<https://colab.research.google.com/drive/1P9PPxzJu5PAVWx1cW0PJCN6-hIZx9ydT?usp=sharing>

OUTPUT SCREENSHOTS:-



test_image.jpg



```
--- Inference Result ---  
Predicted Class: Airplane  
Confidence Score: 99.97%
```

EXPLANATION:-

Part 1: Model Training Pipeline

- **Data Prep & Augmentation:** It downloads the CIFAR-10 dataset and filters out a 3-class subset (**Airplanes, Automobiles, and Birds**). It resizes images to 224×224 pixels and applies random flips and rotations to prevent overfitting.
- **Transfer Learning Configuration:** It loads a pre-trained **ResNet18** model. By freezing the convolutional layers (`param.requires_grad = False`), it preserves generic visual patterns (edges, shapes) learned from millions of ImageNet samples.
- **Custom Head Training:** It swaps the final layer out for a custom linear head (`nn.Linear`) mapped to our 3 target classes. Only this new head is optimized using Cross-Entropy Loss and the Adam optimizer over 5 epochs.
- **Metrics & Export:** It monitors loss and accuracy curves for both training and validation loops, rolls back to the best performing epoch weights, and exports them cleanly into `resnet18_cifar_subset.pth`.

Part 2: Prediction / Inference Script

- **Structure Reconstruction:** The script initializes a blank ResNet18 model, alters its final linear head to accommodate 3 classes, and populates it with your saved `.pth` model weights.
- **Evaluation Mode:** It locks down the network into evaluation state (`model.eval()`), disabling training mechanics like dropout or batch normalization updates.
- **Tensor Processing:** It loads an external image (`test_image.jpg`), converts it to an RGB format, resizes it, applies ImageNet

normalization matrix rules, and stretches it into a $4 \times \text{D}$ batch tensor format (`unsqueeze(0)`).

- **Softmax Prediction:** Running a lightning-fast forward pass inside a `torch.no_grad()` memory block, it routes logits through a `softmax` function to output clear class confidence scores.